

Set up MobSF

Privacy Analysis of Augmented Reality Glasses

1 Set Up before starting MobSF

1. Make sure to have Docker installed on your system. The instructions for downloading on an Ubuntu machine can be found below in the section 'Install Docker'.
2. Get the apk file of the app from the phone, if not done already, the instructions can be found below in the section 'Get APK'.
3. Open a terminal window on your computer.
4. Download the latest MobSF docker image with the command:

```
sudo docker pull opensecurity/mobile-security-framework-mobsf:latest
```

5. To start a dynamic analysis, an emulator is needed. Note that there are certain versions that are not supported by MobSF. The instructions to set up an emulator using 'Genymotion Android VM' can be found below in the section 'Set Up Emulator'. Information on other supported versions and emulators can be found in the link:
https://mobsf.github.io/docs/#/dynamic_analyzer

2 Static Analysis

1. Start MobSF for static analysis in Docker using the command which starts the MobSF server and maps it to port 8000 on your computer:

```
sudo docker run -it --rm \
    -p 8000:8000 \
    opensecurity/mobile-security-framework-mobsf:latest
```

Wait till you see in the output in the terminal indicating that the server has started and is listening on a specific port (8000), it should look like:

```
[2024-11-07 17:54:02 +0000] [1] [INFO] Starting gunicorn 23.0.0
[2024-11-07 17:54:02 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
[2024-11-07 17:54:02 +0000] [1] [INFO] Using worker: gthread
[2024-11-07 17:54:02 +0000] [125] [INFO] Booting worker with pid: 125
```

2. Open a browser and access the MobSF web interface by entering

```
http://127.0.0.1:8000
```

3. The credentials for MobSF, by default, is:

```
username: mobsf
password: mobsf
```

4. Drag and drop your apk file called **base.apk** onto the page and wait for MobSF to create a static analysis. This can take a couple minutes, depending on your app it may take longer.
5. To stop press Ctrl+C in the terminal and close the tab.

3 Static + Dynamic Analysis

1. For Linux based systems, make sure that your Docker version is at least 20.04. To check your docker version, use

```
docker --version
```

2. Start an emulator. If none are set up yet, the instructions to set up an emulator using 'Genymotion Android VM' can be found below in the section 'Emulator'. To start Genymotion navigate to the Genymotion directory then start it:

```
cd ~/genymotion
./genymotion
```

3. Find the adb device identifier. If you are using Genymotion Android VM, you can also find the identifier on the top of your emulated device. If the emulator is running, you can use the command:

```
adb devices
```

The output should look like:

```
List of devices attached
127.0.0.1:6555  device
```

If you do not see a device and your emulator is running, connect your emulator using:

```
adb connect 127.0.0.1:6555
```

4. Start MobSF for dynamic analysis in Docker. Linux based systems can use the following command and replace 127.0.0.1:6555 with the device identifier found.

The following command is what worked for me:

```
sudo docker run -it --rm \
  --network="host" \
  -p 8000:8000 \
  -p 1337:1337 \
  -e MOBSF_ANALYZER_IDENTIFIER=127.0.0.1:6555 \
  --add-host=host.docker.internal:127.0.0.1 \
  opensecurity/mobile-security-framework-mobsf:latest
```

The following command is the original command for Linux systems:

```
sudo docker run -it --rm \
  -p 8000:8000 \
  -p 1337:1337 \
  -e MOBSF_ANALYZER_IDENTIFIER=127.0.0.1:6555 \
  opensecurity/mobile-security-framework-mobsf:latest
```

The original command for the dynamic analysis and the commands for other systems can be found in the link:

https://mobsf.github.io/docs/#/running_mobsf_docker?id=dynamic-analysis

This starts MobSF for dynamic analysis in Docker which starts the MobSF server the two -p is for the static and dynamic analysis respectively. If you receive errors, look below in the section 'Errors encountered' to see if I also had that problem and how I solved it. If not, wait till you see in the output in the terminal indicating that the server has started and is listening on a specific port (8000), it should look like:

```
[2024-11-09 23:27:51 +0000] [1] [INFO] Starting unicorn 23.0.0
[2024-11-09 23:27:51 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
[2024-11-09 23:27:51 +0000] [1] [INFO] Using worker: gthread
[2024-11-09 23:27:51 +0000] [125] [INFO] Booting worker with pid: 125
```

5. Open a browser and access the MobSF web interface by entering

`http://127.0.0.1:8000`

6. The credentials for MobSF, by default, is:

```
username: mobsf
password: mobsf
```

7. Drag and drop your apk file called **base.apk** onto the page and wait for MobSF to create a static analysis. This can take a couple minutes, depending on your app it may take longer.
8. To run a dynamic analysis, press 'start dynamic analysis' on the left bar, which should lead you to a different page.
 - If you see errors, look below in the section 'Errors encountered' to see if I also had that problem and how I solved it.
 - Otherwise, you should see your app and press the green button that says, 'start dynamic analysis' to interact with your app and do an analysis.
9. To stop, close the tab and press Ctrl+C in the terminal.

4 Install Docker

There are some prerequisites and limitations before installing Docker:

1. Firewall limitations: When using ufw or firewalld, exposing container ports with Docker bypasses these firewall rules. Docker only supports iptables-nft and iptables-legacy but not rules created with nft. Ensure any firewall rules are created with iptables or ip6tables and added to the DOCKER-USER chain.
2. OS Requirements: Docker Engine requires a 64-bit version of Ubuntu Noble 24.04 (LTS), Jammy 22.04 (LTS), or Focal 20.04 (LTS), and is compatible with x86_64, armhf, arm64, s390x, and ppc64le architectures.

I installed docker using the apt repository. Steps to install docker using the apt repository:

1. Uninstall old versions or conflicting packages of Docker if there is any using the command though I did not have any:

```
for pkg in docker.io docker-doc docker-compose docker-compose-v2
podman-docker containerd runc; do sudo apt-get remove $pkg; done
```

2. Set up Docker's apt repository first by adding Dock's official GPG key:

```
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

3. Add the repository to apt sources:

```
echo
"deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" |
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
```

4. Install the latest Docker packages:

```
sudo apt-get install docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

To verify that the installation was successful run:

```
sudo docker run hello-world
```

You should see something like:

```
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
...
```

The following link shows more detail as well as other methods for installation for Ubuntu:

<https://docs.docker.com/engine/install/ubuntu/>

5 Get APK

1. Connect your phone to your computer via USB and open a terminal window.
2. First you need to know the package name of the app. The name of the Meta View app is called:

```
com.facebook.stella
```

If you want to look for another app, use the following command in the terminal that outputs the list of package names:

```
adb shell pm list packages
```

You can try to filter the names if you have an idea what may be related to (also the company), but know that the name of the package can be different from the name of the app (as seen for the Meta View app):

```
adb shell pm list packages | grep -i "related_name_1 \| related_name_2"
```

Or you can filter by the installation date, using:

```
adb shell pm list packages -3
```

3. After finding out the name, you are able to get the full path name of the apk file. The following is an example for the Meta View app, for a different app, change the name of the package to the one you want:

```
adb shell pm path com.facebook.stella
```

The output should look something like this:

```
package: /data/app/~~RN3FgfL6QR9KkBav3MNe4g==/com.facebook.stella-i-Y3XkQ  
Ks4B2DUrcEwHUBQ==/base.apk
```

4. With the path name, pull the apk file from the phone to your current working directory, using:

```
abd pull /data/app/~~RN3FgfL6QR9KkBav3MNe4g==/com.facebook.stella-i-Y3XkQ  
Ks4B2DUrcEwHUBQ==/base.apk
```

To pull the apk file to a specific directory, use:

```
abd pull path_you_just_got path_you'd_like_to_pull_to
```

You should see a file with the name in your selected directory:

```
base.apk
```

6 Set Up Emulator

MobSF supports Genymotion Android VM with x86, x86_64 architectures, Android versions 4.1 to 11.0, and upto API 30. The Meta View App requires at least Android version 10.0. Most Android phones use ARM architecture, so an ARM translator is needed. I managed to find a translator for Android version 11.0. In conclusion, we are using the Genymotion Android VM of version 11.0 alongside an ARM translator.

1. Download Genymotion desktop from the link:
<https://www.genymotion.com/product-desktop/download/>
2. Navigate to where the .bin file downloaded to.
3. Give run permissions to the file so it becomes executable. Change the * to the rest of the name for your downloaded .bin file:

```
chmod +x genymotion-*.bin
```

4. Execute the file to start the installation, again changing the * to suit your .bin file:

```
./genymotion-*.bin
```

5. Navigate to the Genymotion installation directory:

```
cd ~/genymotion
```

6. Start Genymotion by:

```
./genymotion
```

7. Genymotion should automatically connect to ADB, and check this by using:

```
adb devices
```

If it is connected, the output should look like:

```
List of devices attached
127.0.0.1:6555  device
```

Otherwise, if the emulator does not appear, link the emulator by using the command below but change the identifier to that of your emulator. The identifier for Genymotion can be found on the top of the running emulated device:

```
adb connect 127.0.0.1:6555
```

8. Install a translator for the emulator if the APK you downloaded is from an ARM-based device (mine was from a Redmi Note 8 which has an arm64 architecture). If it is already x86 architecture, then the rest of the steps are not needed. The translator I used is from:

https://github.com/niizam/Genymotion_A11_libhoudini?tab=readme-ov-file

9. Make sure to open and start the Android emulator.
10. Open a terminal and make sure that the emulator is listed using

```
adb devices
```

11. By default, the /system directory is mounted as read-only. In order to make changes, remount it as read-write;

```
adb root
adb remount
```

12. Open a shell session on the emulator

```
adb shell
```

13. We need to edit the file /system/build.prop. First, navigate to the /system directory:

```
cd /system
```

14. Open the 'build.prop' file using text editors such as vi or echo, for example:

```
vi build.prop
```

15. Press **i** to enter insert mode.

16. Copy and paste the following lines into the file:

```
ro.product.cpu.abi=x86_64,x86,arm64-v8a,armeabi-v7a,armeabi
ro.product.cpu.abi32=x86,armeabi-v7a,armeabi
ro.product.cpu.abi64=x86_64,arm64-v8a
ro.vendor.product.cpu.abi=x86_64,x86,arm64-v8a,armeabi-v7a,armeabi
ro.vendor.product.cpu.abi32=x86,armeabi-v7a,armeabi
ro.vendor.product.cpu.abi64=x86_64,arm64-v8a
ro.odm.product.cpu.abi=x86_64,x86,arm64-v8a,armeabi-v7a,armeabi
ro.odm.product.cpu.abi32=x86,armeabi-v7a,armeabi
ro.odm.product.cpu.abi64=x86_64,arm64-v8a
ro.dalvik.vm.native.bridge=libhoudini.so
ro.enable.native.bridge.exec=1
ro.enable.native.bridge.exec64=1
ro.dalvik.vm.isa.arm=x86
ro.dalvik.vm.isa.arm64=x86_64
ro.zygote=zygote64_32
```

17. When finished, press the following in order:

```
ESC
:wq
Enter
```

18. Now repeat step 13~17, but with the file `/system/vendor/build.prop`.

19. Download libhoudini from the following link:

https://github.com/niizam/Genymotion_A11_libhoudini/releases/download/1.0/system.zip

20. Drag and drop the system.zip you downloaded onto the emulator you have opened, literally onto the phone screen seen on the computer.

21. Restart the emulator.

22. To get out of adb, use:

```
exit
```

7 Errors Encountered

- MobSF cannot detect a running Android emulator

- How the Error looks like:

```
Android Runtime not found!
```

```
MobSF cannot find android instance identifier. Make sure that an android instance is
```

```
If this error persists, Please set ANALYZER\_IDENTIFIER in /home/mobsf/.MobSF/config.
```

- Verify that the emulator is running and connected, by:

```
adb devices
```

- * If you see:

```
List of devices attached
127.0.0.1:6555 device
```

Try to set the environment variable to the identifier of the emulator using:

```
export MOBSF_ANALYZER_IDENTIFIER=127.0.0.1:6555
```

* If not, try to connect the emulator:

```
adb connect 127.0.0.1:6555
```

– Also make sure to run MobSF with the environment variable:

```
sudo docker run -it --rm \
  -p 8000:8000 \
  -p 1337:1337 \
  -e MOBSF_ANALYZER_IDENTIFIER=127.0.0.1:6555 \
  --add-host=host.docker.internal:host-gateway \
  opensecurity/mobile-security-framework-mobsf:latest
```

- MobSF is trying to restart the ADB daemon as root but cannot connect to the emulator at host.docker.internal:5555.

– How the Error looks like:

```
adb: unable to connect for root: device
'host.docker.internal:5555' not found
...
subprocess.CallProcessError: Command '['/usr/bin/adb', '-s', 'host.docker.internal:
[ERROR] 07/Nov/2024 21:55:00 - Dynamic Analysis Failed.
[ERROR] 07/Nov/2024 21:55:00 - Internal Server Error: /android_dynamic/972f2d1ca570d6
```

Or if it looks like:

```
* daemon not running; starting now at tcp:5037
* daemon started successfully
Connection refused
```

– Check the IP address for the host and start MobSF using this IP address for the MOBSF_ANALYZER_IDENTIFIER. The IP address can be obtained by:

```
ip route | grep docker0
```

– Change the command when starting MobSF to:

```
sudo docker run -it --rm \
  --network="host" \
  -p 8000:8000 \
  -p 1337:1337 \
  -e MOBSF_ANALYZER_IDENTIFIER=127.0.0.1:6555 \
  --add-host=host.docker.internal:127.0.0.1 \
  opensecurity/mobile-security-framework-mobsf:latest
```

The `--network="host"` option allows the Docker container to share the host machine's network stack.

- MobSF is trying to connect to host.docker.internal:6555 despite specifying 127.0.0.1:6555 as the MOBSF_ANALYZER_IDENTIFIER (from our initial command).

– How the Error looks like:

```
[ERROR] 09/Nov/2024 12:02:03 - failed to connect to 'host.docker.internal:6555': Conn
[ERROR] 09/Nov/2024 12:02:03 - Cannot Connect to host.docker.internal:6555
[ERROR] 09/Nov/2024 12:02:03 - Internal Server Error: /android_dynamic/972f2d1ca570d6
```

– Change the command to start MobSF to:

```
sudo docker run -it --rm \
  --network="host" \
  -p 8000:8000 \
  -p 1337:1337 \
  -e MOBSF_ANALYZER_IDENTIFIER=127.0.0.1:6555 \
  --add-host=host.docker.internal:127.0.0.1 \
  opensecurity/mobile-security-framework-mobsf:latest
```

By changing the host redirects any reference to host.docker.internal within the container to 127.0.0.1

- MobSF successfully connected to the emulator and prepared the environment, but encountered an issue installing the APK.

- How the Error looks like:

```
[ERROR] 09/Nov/2024 12:09:23 - This APK cannot be installed. Is this APK compatible t
adb install failed
```

```
[ERROR] 09/Nov/2024 12:09:23 - Internal Server Error: /android_dynamic/972f2d1ca570d6
```

- This could be due to the APK compatibility. Make sure that all the versions and architecture are compatible between MobSF, the app, and the emulator. I, for example, had to use Genymotion with Android version 11.0 with an ARM translator.

- ADB detects multiple connected devices, so it doesn't know which one to target.

- How the Error looks like:

```
adb: unable to connect for root: more than one device/emulator
```

```
adb: more than one device/emulator
```

- * You can specify the device by adding the -s option with the device identifier.

- * Or you can also try to disconnect the other to avoid conflicts by using the command:

```
adb disconnect host.docker.internal:6555
```